

30-Day Java DSA Placement Roadmap — Redesigned

Optimized for Placements & Internships | Java | 5-7 hrs/day | B.Tech 7th Semester Prep

Critical Review of Original Roadmap

What was wrong with the original plan:

- **LeetCode started too late (Day 12)** — Pattern recognition needs to start on Day 4 alongside your first heavy topic (Sorting). Placements test patterns, not theory.
- **OOP given a full day + mini-project** — OOP concepts take 3-4 hours max for placement purposes. Interviewers rarely test mini-projects; they test concepts + code.
- **Trees underweighted** — Trees/BST are the #1 most tested topic in product company coding rounds. The original gave them 3 days total. They need 5-6 days.
- **Custom HashMap implementation overemphasized** — Worth understanding conceptually (30 min). Not worth a full day. Use Java's built-in HashMap and solve 8-10 HashMap problems instead.
- **Problem count too low** — 75 problems in 30 days with 5-7 hrs/day is too conservative. Realistic target with this schedule: 120-150 problems.
- **No Binary Search track** — Binary Search appears in 30-40% of Easy/Medium rounds. It was missing entirely.
- **Buffer days too spread out** — Better to have focused revision blocks at week ends than daily buffers.

Topic Priority Classification

Priority	Topics	Why
MUST MASTER	Arrays, Strings, Recursion, Binary Search, Sorting (Merge/Quick), Linked List, Stack, Queue, HashMap, Trees, BST	Appear in 80%+ of coding rounds. Cannot skip. Must solve 8-10 problems each minimum.
IMPORTANT	Two Pointers, Sliding Window, Graphs (BFS/DFS), Heap/PriorityQueue, OOP concepts	Appear in 40-60% of rounds. Medium difficulty rounds test these.
NICE TO HAVE	Trie, Dijkstra, Segment Tree, Custom HashMap, Advanced Graph algorithms, Dynamic Programming (beyond basics)	Senior roles or product companies. Not needed for first placement/internship round.

Should you implement a custom HashMap? Spend 30 minutes understanding the concept (hash function, array of linked lists for chaining, load factor). That is enough for interviews — they may ask "how does HashMap work internally?" but they will NOT ask you to code one from scratch in a timed OA. Instead, use Java's built-in HashMap and solve 8-10 pattern problems: frequency counters, two-sum, anagram detection, group by key, sliding window with map. That is 10x more valuable.

Daily Schedule Template (5-7 hrs)

Block	Time	Duration	Activity
Morning	7:00-8:30 AM	1.5 hr	Watch concept video + take notes + implement core DS from scratch in Java
Afternoon	11:00 AM-1:00 PM	2 hr	Solve 4-6 LeetCode problems on today's topic. Timer: 20 min per Easy, 35 min per Medium. If stuck after timer, read editorial, understand, re-implement.
Evening	4:00-6:00 PM	2 hr	Solve 2-3 more problems. Review yesterday's solutions. Write approach in comments.
Night	9:30-10:00 PM	30 min	GitHub commit. Update README daily log. Push code. Review 1 problem you got wrong.

GitHub Repository Structure

Repository name: DSA-Java-Placement-Prep

DSA-Java-Placement-Prep/

├─ README.md

├─ Arrays/

├─ Notes.md

├─ ArrayListImpl.java

├─ TwoSum.java

└─ MaxSubarrayKadane.java

├─ Strings/

├─ Notes.md

└─ AnagramCheck.java

├─ Recursion/

├─ BinarySearch/

├─ Sorting/

├─ MergeSort.java

└─ QuickSort.java

├─ LinkedList/

├─ SinglyLinkedList.java

└─ DoublyLinkedList.java

├─ Stack/

├─ Queue/

├─ Trees/

├─ BinaryTree.java

├─ Traversals.java

└─ LevelOrderBFS.java

├─ BST/

└─ BSTImpl.java

├─ Hashing/

├─ Graphs/

└─ MockInterviews/

├─ Week2_Mock.md

GitHub README Daily Log Template

```
## Day [N] – [Topic Name] – [Date]

**Problems Solved Today:** X | **Total Problems:** Y / 120

| # | Problem | Difficulty | Status | Notes |
|---|-----|-----|-----|-----|
| 1 | Two Sum | Easy | ✓ Solved | HashMap O(n) approach |
| 2 | Valid Parentheses | Easy | ✓ Solved | Stack approach |
| 3 | Merge Intervals | Medium | ⚠ Hint used | Sorting + merge logic |

**Key Lesson:** [One sentence: what pattern or concept clicked today]

**Struggled With:** [Honest note on what was hard]

**Tomorrow's Goal:** [Topic + target problems]

**GitHub Streak:** Day N/30 ✓
```

WEEK 1 — Foundations: Arrays, Strings, Recursion, Sorting + Binary Search

Week 1 Milestone: 35+ problems solved | Merge Sort + Quick Sort implemented | Binary Search implemented | LeetCode streak started | GitHub streak active

Day	Topic	Implement	LeetCode Problems	Target	GitHub
Day 1	Arrays & Strings Warm-up (You already know these — use them to build habits)	Write utility methods: reverse array, rotate array, find duplicates, check palindrome string	Easy Two Sum (#1) Easy Best Time to Buy/Sell Stock (#121) Easy Contains Duplicate (#217) Easy Valid Anagram (#242) Easy Reverse String (#344)	5 problems	Repo setup, Day1 folder, README template
Day 2	Arrays — Patterns Two Pointers, Prefix Sum	Implement prefix sum array. Implement two-pointer template.	Easy Squares of Sorted Array (#977) Easy Move Zeroes (#283) Easy Running Sum of Array (#1480) Med 3Sum (#15) Med Max Subarray - Kadane's (#53)	5 problems	Day2 commit
Day 3	Strings — Patterns Sliding Window, String Builder	Implement: check anagram, longest substring tracker, string reversal by words	Easy Valid Palindrome (#125) Easy First Unique Character (#387) Easy Longest Common Prefix (#14) Med Longest Substring Without Repeating (#3) Med Group Anagrams (#49)	5 problems	Day3 commit
Day 4	Binary Search MUST MASTER Missed in original plan	Implement binary search (iterative + recursive). Implement lower_bound / upper_bound templates.	Easy Binary Search (#704) Easy Search Insert Position (#35) Easy First Bad Version (#278) Med Find Minimum in Rotated Sorted Array (#153) Med Search in Rotated Sorted Array (#33)	5 problems	Day4 commit
Day 5	Recursion Basics Call stack, base case, recursion tree	Factorial, Fibonacci, Sum of digits, Power(x,n), Print all subsets of array	Easy Reverse String (recursive) (#344) Easy Power of Two (#231) Easy Climbing Stairs (#70) Med Subsets (#78) Med Letter Combinations of Phone Number (#17)	5 problems	Day5 commit
Day 6	Merge Sort + Quick Sort Divide & conquer — implement both	Merge Sort from scratch (Java). Quick Sort from scratch (Java). Compare time complexities.	Easy Sort Array by Parity (#905) Med Sort Colors (#75 — Dutch Flag) Med Merge Intervals (#56) Med Kth Largest Element in Array (#215)	4 problems	Day6 commit
Day 7	Week 1 Revision + Backtracking intro	Implement N-Queens (classic backtracking). Re-implement Binary Search from memory.	Med Permutations (#46) Med Combination Sum (#39) Easy Revisit 2 problems you struggled with this week Med Word Search (#79)	4-5 problems	Weekly summary commit + README week review

WEEK 2 — Linear Data Structures: Linked List, Stack, Queue + OOP (condensed)

Week 2 Milestone: 35+ problems solved (cumulative 70+) | Singly LL + Stack + Queue fully implemented | OOP concepts understood | Mock Interview 1 completed

Day	Topic	Implement	LeetCode Problems	Target	GitHub
Day 8	OOP in Java — Condensed Classes, Constructors, Inheritance, Polymorphism, Abstraction, Interfaces — ONLY	Write: Animal hierarchy (Inheritance), Shape interface (Abstraction + Polymorphism). No large projects.	Easy Design Parking System (#1603) Easy Design HashSet (#705) Easy Number of Students Unable to Eat Lunch (#1700)	3 problems	Day8 commit + OOP Notes.md
Day 9	Java Collections Framework ArrayList, LinkedList, Stack, Queue, PriorityQueue in Collections	Sort custom objects using Comparator. Use PriorityQueue for top-K problems.	Easy Kth Largest Element in Stream (#703) Med Top K Frequent Elements (#347) Med Sort Characters by Frequency (#451) Easy Last Stone Weight (#1046)	4 problems	Day9 commit
Day 10	Linked List — Singly MUST MASTER	Implement from scratch: insert at head/tail/position, delete by value, reverse, detect loop (Floyd's), find middle	Easy Reverse Linked List (#206) Easy Middle of Linked List (#876) Easy Linked List Cycle (#141) Med Remove Nth Node from End (#19)	4 problems	Day10 commit
Day 11	Linked List — Advanced + Doubly LL	Implement Doubly Linked List. Implement merge two sorted lists (no extra space).	Med Merge Two Sorted Lists (#21 — Easy but pattern) Med Reorder List (#143) Med Add Two Numbers (#2) Med Intersection of Two Linked Lists (#160)	4 problems	Day11 commit
Day 12	Stack MUST MASTER — most frequent in OAs	Implement Stack using array AND linked list. Implement Min Stack.	Easy Valid Parentheses (#20) Easy Min Stack (#155) Med Daily Temperatures (#739) Med Next Greater Element II (#503) Med Evaluate Reverse Polish Notation (#150)	5 problems	Day12 commit
Day 13	Queue + Deque FIFO, circular queue, sliding window with deque	Implement Queue (array + LL). Implement circular queue. Use ArrayDeque for sliding window max.	Med Implement Queue using Stacks (#232) Med Sliding Window Maximum (#239) Med Number of Recent Calls (#933) Easy First Unique Character in Stream	4 problems	Day13 commit
Day 14	Mock Interview 1 + Week 2 Revision	Mock Interview Format (2 hrs): Pick 1 Easy + 1 Medium from LC. Set a 45-min timer each. No hints. Write solution + explain your approach in comments as if talking to an interviewer. Mock 1 suggested problems: Valid Parentheses (Easy, Stack) + LRU Cache (Medium, if ambitious) OR Reverse Linked List + Merge Intervals. After mock: 2 hr revision — re-implement Linked List and Stack from memory without looking at code.	Mock + 3 revision problems	Mock Interview notes committed to MockInterviews/Week2_Mock.md	

WEEK 3 — Trees, BST, HashMap — The Most Important Week

This is the most critical week. Trees + BST appear in nearly every product company coding round. Do NOT rush. If you must choose between speed and depth here, choose depth. Aim for 30+ problems this week.

Week 3 Milestone: 30+ problems solved (cumulative 100+) | Binary Tree fully implemented with all traversals | BST implemented with insert/search/delete | HashMap patterns mastered

Day	Topic	Implement	LeetCode Problems	Target	GitHub
Day 15	HashMap — Patterns MUST MASTER — No custom implementation needed	Conceptual: understand how HashMap works internally (5 mins). Practice patterns: frequency counter, two-sum variant, group by key.	Easy Two Sum (#1 — redo with HashMap) Easy Ransom Note (#383) Easy Isomorphic Strings (#205) Med Subarray Sum Equals K (#560) Med Longest Consecutive Sequence (#128)	5 problems	Day15 commit + Hashing Notes.md
Day 16	HashSet + Sliding Window with Hashing	Solve problems using HashSet. Master sliding window + HashMap combo.	Easy Contains Duplicate II (#219) Med Minimum Window Substring (#76) Med Find All Anagrams in String (#438) Med 4Sum II (#454)	4 problems	Day16 commit
Day 17	Binary Tree — Basics + Traversals MUST MASTER	Implement BinaryTree class. Implement: inorder, preorder, postorder (recursive + iterative). Implement level order (BFS using queue).	Easy Binary Tree Inorder Traversal (#94) Easy Maximum Depth of Binary Tree (#104) Easy Symmetric Tree (#101) Med Binary Tree Level Order Traversal (#102) Med Binary Tree Zigzag Level Order (#103)	5 problems	Day17 commit + Trees/Notes.md
Day 18	Binary Tree — Interview Questions Height, diameter, paths, views	Implement: height of tree, diameter of tree, left/right view, check balanced tree.	Easy Balanced Binary Tree (#110) Easy Path Sum (#112) Med Diameter of Binary Tree (#543 — Easy but important) Med Binary Tree Right Side View (#199) Med Path Sum II (#113)	5 problems	Day18 commit
Day 19	BST — Full Implementation MUST MASTER	Implement BST from scratch: insert, search, delete (3 cases), inorder (gives sorted output), find min/max, floor/ceil.	Easy Search in BST (#700) Easy Insert into BST (#701) Med Delete Node in BST (#450) Med Kth Smallest in BST (#230) Med Validate Binary Search Tree (#98)	5 problems	Day19 commit + BST/Notes.md
Day 20	BST + Tree Mixed Interview Problems	No new implementation — focus entirely on solving interview-style problems.	Med Lowest Common Ancestor of BST (#235) Med Lowest Common Ancestor of BT (#236) Med Construct BT from Preorder + Inorder (#105) Med Flatten BT to Linked List (#114) Easy Same Tree (#100)	5 problems	Day20 commit
Day 21	Trees Revision + Extra Practice	Re-implement BST delete from memory. Write tree traversal iteratively (stack-based).	Med Binary Tree Maximum Path Sum (#124) Med Count Good Nodes in BT (#1448) Easy Merge Two Binary Trees (#617) Med Serialize and Deserialize BT (#297 — Hard, optional stretch)	3-4 problems	Weekly summary commit

Week 4 Milestone: 20+ new problems (cumulative 120+) | Graph BFS/DFS implemented | Final mock interview completed | Day 30 scorecard filled

Day	Topic	Implement	LeetCode Problems	Target	GitHub
Day 22	Graphs — Representation + BFS IMPORTANT	Implement: adjacency list (HashMap + List), BFS using queue, print all connected nodes.	Med Number of Islands (#200) Med Flood Fill (#733 — Easy, warm-up) Med Rotting Oranges (#994) Med Clone Graph (#133)	4 problems	Day22 commit + Graphs/Notes.md
Day 23	Graphs — DFS + Cycle Detection	Implement DFS (recursive + iterative). Implement cycle detection (directed + undirected).	Med Course Schedule (#207 — cycle detection DAG) Med Number of Connected Components (#323) Med Pacific Atlantic Water Flow (#417) Easy Find if Path Exists in Graph (#1971)	4 problems	Day23 commit
Day 24	Heap / Priority Queue + Greedy IMPORTANT — often missed in prep	Use Java PriorityQueue. Solve top-K pattern problems. Understand min-heap vs max-heap.	Med Top K Frequent Elements (#347) Med K Closest Points to Origin (#973) Med Task Scheduler (#621) Easy Last Stone Weight (#1046)	4 problems	Day24 commit
Day 25	Trie — Conceptual + Basic Implementation NICE TO HAVE	Implement basic Trie: insert, search, startsWith. Understand where it's used (autocomplete, prefix problems).	Med Implement Trie (#208) Med Word Search II (#212 — stretch, use Trie) Med Design Add and Search Words (#211)	2-3 problems	Day25 commit
Day 26	Mixed Revision — Arrays + Strings + Binary Search	No new topics. Re-solve 2-3 Week 1 problems from memory (no peeking).	Med 2 new Medium problems on arrays/strings you haven't done Easy 3 Easy problems to build confidence	5 problems	Revision commit
Day 27	Mixed Revision — Linked List + Stack + Queue	Re-implement Singly Linked List from scratch (time yourself: target under 20 minutes).	Med LRU Cache (#146) Med 2 unsolved Stack/Queue Mediums from backlog	3-4 problems	Revision commit
Day 28	Mixed Revision — Trees + BST + Hashing	Re-implement BST delete from memory. Write all traversals without notes.	Med 3-4 Tree problems from backlog or unsolved Easy 2 HashSet/HashMap easy problems	5 problems	Revision commit
Day 29	Mock Interview 2 — Full Simulation	Format: 2 rounds × 45 minutes Round 1: 1 Easy + 1 Medium (Arrays/Strings/HashMap) Round 2: 1 Easy + 1 Medium (Trees/Graphs) Suggested problems: R1: Valid Anagram + Subarray Sum Equals K R2: Binary Tree Level Order + Number of Islands After mock: write a 1-page self-assessment in MockInterviews/Week4_Mock.md — what you solved, what you struggled with, time taken per problem.		4 mock problems	Mock results committed
Day 30	Final Review + Scorecard Day	Review all Notes.md files. List top 5 unsolved problems to revisit in semester.	Med 3-5 mixed difficulty problems of your choice Final GitHub cleanup: ensure every folder has Notes.md	3-5 problems	Final push. Pin repo. Update README with summary stats.

Problem Count Targets

Week	New Problems	Cumulative	Difficulty Mix
Week 1 (Days 1-7)	33-35	35	60% Easy / 40% Medium
Week 2 (Days 8-14)	25-30	65	50% Easy / 50% Medium
Week 3 (Days 15-21)	30-35	100	40% Easy / 60% Medium
Week 4 (Days 22-30)	20-25	120-130	30% Easy / 70% Medium
TOTAL	Minimum: 75 Target: 120 Stretch: 150		End state: 70% Medium comfort

Recovery Plans

Scenario	Action Plan
Miss 3 days	Don't try to catch up all topics. Identify the 1 missed topic that matters most (check MUST MASTER list). Spend 1 day on it. Skip or compress NICE TO HAVE topics. GitHub: even a README update or 1-problem commit counts as a streak day. Target: stay within 10 problems of your weekly goal.
Miss 5 days	Drop: Trie (Day 25), Heap deep dive, custom implementations of lower-priority topics. Merge: Days 22+23 into 1 day (BFS + DFS basics only — just enough to solve Number of Islands). Keep: All Week 1-3 topics intact. Adjust final target to 90 problems. GitHub minimum: 1 commit every 2 days during travel/break.
Miss 7+ days	Priority survival list — solve only these if time is critically short: Arrays + Strings (Days 1-3) → Binary Search (Day 4) → Linked List (Days 10-11) → Stack (Day 12) → HashMap (Day 15) → Binary Tree (Days 17-18) → BST (Day 19) → Graph BFS (Day 22). That is 8 core topic days. Compress into 5 days if needed. Solve 10 problems per day. Target 60+ minimum. GitHub: 1 commit per topic even if it's just pushing your solution file.

Golden Rule for GitHub streak: On a missed or travel day, write a one-line entry in README.md: "Day [N] — travel. Plan to cover [topic] tomorrow." That counts. Push it. Protect the streak even symbolically.

Weekly Milestones

Week	Must Complete	Problem Goal	Implementation Goal
Week 1	Arrays, Strings, Binary Search, Recursion, Merge Sort, Quick Sort	35+ problems	BinarySearch.java, MergeSort.java, QuickSort.java
Week 2	OOP concepts (no big project), ArrayList, Linked List (Singly + Doubly), Stack, Queue	35+ problems (70 cumulative)	SinglyLinkedList.java, Stack.java, Queue.java
Week 3	HashMap patterns, HashSet, Binary Tree (all traversals), BST (full implementation)	30+ problems (100 cumulative)	BinaryTree.java, BST.java with all operations
Week 4	Graph BFS + DFS, Heap/PQ basics, Trie intro, 2 mock interviews, final revision	20+ problems (120 cumulative)	Graph.java, MockInterview notes

Mock Interview Checkpoints

Mock	When	Format	What to Assess
Mock Interview 1	Day 14 (End of Week 2)	2 problems in 90 min (1 Easy + 1 Medium). Solve on paper/IDE without hints.	Can you solve a Linked List or Stack problem end-to-end? Can you explain your approach out loud?
Mock Interview 2	Day 29 (End of Week 4)	4 problems in 90 min (2 Easy + 2 Medium). Simulate real OA conditions.	Can you handle Trees + Graphs + Arrays in one sitting? Time per problem? Confidence level?

Topic-wise LeetCode Quick Reference

Topic	Must-Solve Easy	Must-Solve Medium
Arrays	Two Sum, Contains Duplicate, Best Time to Buy Stock	Max Subarray, 3Sum, Merge Intervals, Product Except Self
Strings	Valid Palindrome, Valid Anagram, Reverse Words	Longest Substring Without Repeat, Group Anagrams, Minimum Window Substring
Binary Search	Binary Search, Search Insert Position, First Bad Version	Find Min in Rotated Array, Search in Rotated Array, Koko Eating Bananas
Recursion/Backtracking	Climbing Stairs, Power of Two	Subsets, Permutations, Combination Sum, Word Search
Linked List	Reverse LL, Middle of LL, LL Cycle	Remove Nth from End, Add Two Numbers, Reorder List
Stack	Valid Parentheses, Min Stack	Daily Temperatures, Next Greater Element, Evaluate RPN
Queue/Deque	Implement Queue w/ Stacks	Sliding Window Maximum, Rotting Oranges
HashMap/HashSet	Two Sum, Ransom Note, Isomorphic Strings	Subarray Sum = K, Longest Consecutive, Group Anagrams
Binary Tree	Max Depth, Symmetric Tree, Path Sum, Inorder Traversal	Level Order Traversal, Zigzag Traversal, Right Side View, Path Sum II
BST	Search in BST, Insert into BST	Validate BST, Kth Smallest, Delete Node, LCA of BST
Graphs	Flood Fill, Find Path in Graph	Number of Islands, Course Schedule, Rotting Oranges, Clone Graph

Day 30 Evaluation Scorecard

Fill this on Day 30. Be honest. Total your score and find your placement readiness level.

Category	Metric	Your Score	Points
Problems Solved	150+ problems solved	___	25 pts
	120-149 problems solved	___	20 pts
	90-119 problems solved	___	15 pts
	Below 90 problems	___	8 pts
Topics Completed	All 13 topics (incl. Graphs + Heap)	___	20 pts
	11-12 topics	___	16 pts
	9-10 topics	___	12 pts
	Below 9 topics	___	6 pts
Implementations	15+ from scratch (LL, Stack, Queue, BST, BT, Graph, Sorting, BS)	___	15 pts
	10-14 implementations	___	10 pts
	Below 10	___	5 pts
GitHub Streak	27-30 day streak	___	15 pts
	20-26 day streak	___	10 pts
	Below 20 days	___	5 pts
Mock Interviews	Both mocks completed — solved 3+ of 4 problems	___	15 pts
	Both mocks completed — solved 2 of 4	___	10 pts
	Only 1 mock or solved fewer than 2	___	5 pts
Medium Problem Confidence	Can solve Medium in under 35 min consistently	___	10 pts
	Can solve most Mediums with some struggle	___	7 pts
	Struggle significantly with Mediums	___	3 pts
TOTAL SCORE		___/100	100 pts max

Scorecard Result

Total Score	Verdict	What It Means
-------------	---------	---------------

Below 45	● Not Ready	Need 2–4 more weeks of focused practice. Focus on MUST MASTER topics only. Do 10 problems/day for 2 weeks.
45–64	● Somewhat Ready	Can clear initial HackerRank/CodeChef style OAs. Need more Medium exposure before appearing for tier-1 companies. Keep going — you are close.
65–80	● Placement Ready	Solid foundation. Can confidently appear for most service companies and mid-tier product companies. Practice 2–3 more Mediums/day during semester.
81–100	● Strong Candidate	Well prepared for product company OAs and technical interviews. Can target FAANG-adjacent companies. Focus now on DP and system design basics in semester.

Printable 30-Day Checklist

Day	Topic	Video	Concept	Impl.	Problems	GitHub	Status
1	Arrays & Strings Warm-up	☐	☐	☐	__/5	☐	_____
2	Arrays — Two Pointers / Prefix Sum	☐	☐	☐	__/5	☐	_____
3	Strings — Sliding Window	☐	☐	☐	__/5	☐	_____
4	Binary Search	☐	☐	☐	__/5	☐	_____
5	Recursion Basics	☐	☐	☐	__/5	☐	_____
6	Merge Sort + Quick Sort	☐	☐	☐	__/4	☐	_____
7	Backtracking + Week 1 Revision	☐	☐	☐	__/5	☐	_____
8	OOP — Condensed	☐	☐	☐	__/3	☐	_____
9	Java Collections + PriorityQueue	☐	☐	☐	__/4	☐	_____
10	Linked List — Singly	☐	☐	☐	__/4	☐	_____
11	Linked List — Advanced / Doubly	☐	☐	☐	__/4	☐	_____
12	Stack	☐	☐	☐	__/5	☐	_____
13	Queue + Deque	☐	☐	☐	__/4	☐	_____
14	Mock Interview 1 + Week 2 Revision	☐	☐	☐	__/3	☐	_____
15	HashMap Patterns	☐	☐	☐	__/5	☐	_____
16	HashSet + Sliding Window + Hashing	☐	☐	☐	__/4	☐	_____
17	Binary Tree — Basics + Traversals	☐	☐	☐	__/5	☐	_____
18	Binary Tree — Interview Questions	☐	☐	☐	__/5	☐	_____
19	BST — Full Implementation	☐	☐	☐	__/5	☐	_____
20	BST + Tree Mixed Problems	☐	☐	☐	__/5	☐	_____
21	Trees Revision + Extra Practice	☐	☐	☐	__/4	☐	_____
22	Graphs — Representation + BFS	☐	☐	☐	__/4	☐	_____
23	Graphs — DFS + Cycle Detection	☐	☐	☐	__/4	☐	_____
24	Heap / PriorityQueue + Greedy	☐	☐	☐	__/4	☐	_____
25	Trie (Conceptual + Basic)	☐	☐	☐	__/3	☐	_____
26	Revision — Arrays + Strings + BS	☐	☐	☐	__/5	☐	_____
27	Revision — LL + Stack + Queue	☐	☐	☐	__/4	☐	_____
28	Revision — Trees + BST + Hashing	☐	☐	☐	__/5	☐	_____
29	Mock Interview 2 — Full Simulation	☐	☐	☐	__/4	☐	_____
30	Final Review + Scorecard	☐	☐	☐	__/5	☐	_____
TOTAL					___/120	___/30	

Final Message: The gap between students who get placements and those who don't is not intelligence — it is consistency and pattern recognition. 120 solved problems with honest effort, clean GitHub history, and two mock interviews completed will put you ahead of 80% of your batch. Start Day 1 with the mindset that every commit is a data point your recruiter can see. Make it count. Good luck.